SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1063	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:		**Reg. No.:**	
Date:		**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes

Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is incomplete, inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture selected or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and	Diagram is incomplete, unclear, or

	major components, interfaces, and interactions accurately represented.		lacks sufficient detail.	technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions and quality attributes are poorly explained or missing.
Architectural Justification and Technical Presentation (10%)	Provides strong justification for architectural decisions, discusses trade-offs and future enhancements, and presents a well-organized, professional report.	Provides adequate justification with minor gaps; report is well organized.	Limited justification with minimal discussion of trade-offs or future scope; report needs improvement.	Justification is weak or absent; report lacks organization and technical clarity.

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

ARCHITECTURE DESIGN ASSIGNMENT

Personal Expense and Budget Management System

1. Introduction

A Personal Expense and Budget Management System is a software application designed to help users manage income, track daily expenses, organize spending into categories, set budgets, receive alerts, and generate financial reports. Many people find it difficult to understand where their money goes because records may be maintained in notebooks, spreadsheets, and different applications.

The proposed system provides a centralized platform where users can record financial transactions, monitor spending habits, and make informed financial decisions. For example, a user can create a monthly budget of ₹5,000 for food and ₹2,000 for transportation. When spending approaches or exceeds a configured limit, the application can send a notification.

From an Advanced Software Engineering perspective, designing this system requires requirement analysis, selection of a suitable software architecture, identification of components, definition of interactions, and evaluation of quality attributes. A well-designed architecture makes the application secure, reliable, maintainable, and capable of supporting future enhancements.

Objectives of the system:

- Record and manage income and expenses.
- Categorize transactions such as food, transport, education, shopping, and bills.
- Create daily, weekly, and monthly budgets.
- Generate alerts when spending approaches or exceeds budget limits.
- Display financial summaries, charts, and reports.
- Protect financial information through authentication and access control.
- Support future improvements without requiring major changes to the existing system.

2. System Requirement Analysis

Requirement analysis identifies what the software must do and the conditions under which it must operate. It provides the foundation for selecting the architecture and designing the system components.

2.1 Problem Statement

Without a centralized expense management application, users may find it difficult to maintain accurate financial records, identify unnecessary spending, and follow a budget. Manual calculations can introduce errors, and overspending may not be noticed until the end of the month. The proposed application automates transaction recording, category-wise tracking, budget calculations, notifications, and report generation.

2.2 Functional Requirements

ID	Requirement	Description
FR1	Registration and login	Allow users to create accounts and log in securely.
FR2	Income management	Record income amount, source, date, and description.
FR3	Expense management	Add, edit, view, and delete expense records.
FR4	Expense categorization	Classify transactions into food, travel, bills, education, and other categories.
FR5	Budget management	Set spending limits for categories and selected periods.
FR6	Budget monitoring	Calculate actual spending and compare it with budget limits.
FR7	Alert management	Notify users when spending reaches a configured threshold or exceeds the budget.
FR8	Financial reporting	Generate daily, monthly, and yearly summaries, charts, and category-wise reports.
FR9	Search and filtering	Filter transactions by date, category, type, and amount.
FR10	Data management	Store and retrieve transactions securely and maintain consistent records.

2.3 Non-Functional Requirements

Attribute	Requirement	Importance
Performance	Common operations should normally complete within about 2 seconds under the expected workload.	Provides a responsive experience.
Security	Use secure authentication, authorization, encrypted communication, and protected credential storage.	Prevents unauthorized access.
Reliability	Store financial transactions accurately and avoid unintended data loss or duplication.	Maintains trust in records.
Availability	Keep the service accessible with minimal planned downtime, subject to hosting and network availability.	Supports timely access.
Scalability	Support growth in users and transaction volumes.	Accommodates future demand.
Maintainability	Use clear component responsibilities and interfaces.	Makes changes and debugging easier.
Usability	Provide a simple, consistent interface.	Reduces learning time and user errors.
Data integrity	Keep transaction amounts, budget calculations, and account ownership consistent.	Ensures reliable reports.
Portability	Support modern browsers and possible mobile integration.	Expands accessibility.
Privacy	Prevent unauthorized access to users' financial information.	Protects personal data.

2.4 Quality Attributes Influencing Architecture

The most important quality attributes are security, reliability, performance, scalability, and maintainability. Security influences authentication, access-control rules, and database protection. Reliability influences transaction handling, backups, and recovery. Performance influences indexing and report-query design. Scalability influences how backend services and alert processing can be expanded. Maintainability requires the separation of presentation, business logic, and data access. These attributes must be considered together because architectural decisions involve trade-offs.

3. Selection of Software Architecture

3.1 Proposed Architecture: Layered Three-Tier Architecture

The recommended architecture is a layered architecture implemented as a three-tier application. It separates the presentation layer, application and business logic layer, and data access and persistence layer. The presentation layer handles user interaction, the business layer applies financial rules, and the data layer manages persistent storage.

This architecture suits the project because it has clear user-interface operations, financial rules, and database requirements. It provides a practical balance between simplicity, security, maintainability, and future scalability without introducing the operational complexity of a large microservices-based system.

3.2 Architecture Diagram

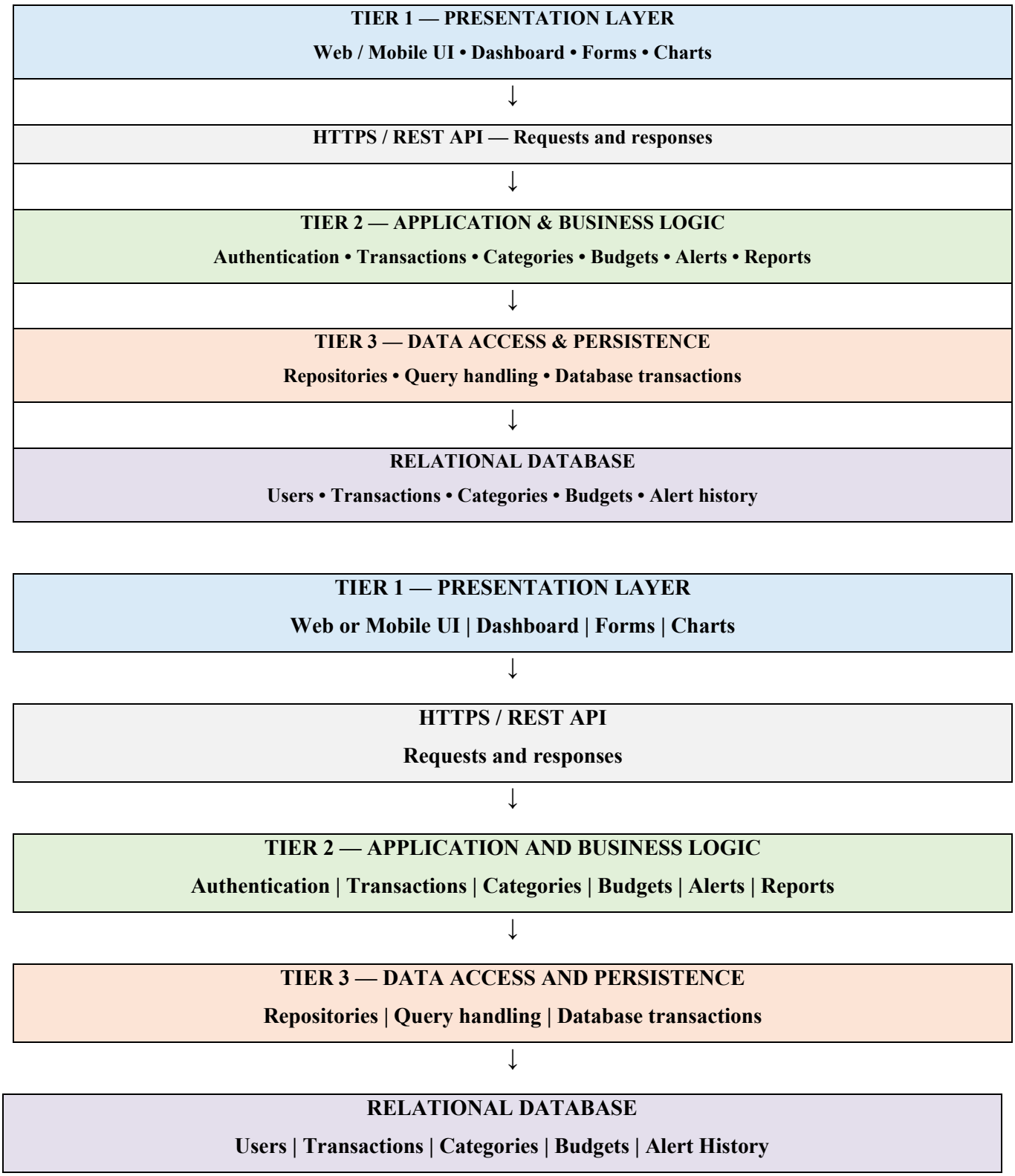


Figure 1. Logical layered architecture of the proposed system.

The user interacts with the presentation layer. Requests are sent through the API to the application layer, where the system validates inputs and applies business rules. The data access layer retrieves or stores records in the relational database. Results are returned to the user interface for display.

3.3 Technical Justification

- Separation of concerns: User-interface code, financial calculations, and database operations remain separate.
- Security: Authentication and authorization can be enforced at the application boundary, and data-access rules can restrict users to their own records.
- Maintainability: Dashboard changes can be made without rewriting budget calculation logic.
- Testability: Budget calculations, expense categorization, and alert thresholds can be tested independently.
- Data consistency: A relational database supports structured relationships and database transactions.
- Cost-effectiveness: The system can initially run as one backend service instead of multiple independently operated services.
- Future scalability: Caching, background jobs, database optimization, and additional application instances can be introduced as demand increases.

3.4 Comparison with Alternative Architectures

Architecture	Advantages	Limitations for this project
Layered architecture	Simple, modular, understandable, and easy to test.	Requires discipline to keep layer boundaries clear.
Microservices	Services can be deployed and scaled independently.	Adds deployment, communication, monitoring, and data-consistency complexity.
Event-driven	Suitable for asynchronous alerts and background processing.	Requires careful event delivery, retries, and ordering.
Client-server	Provides centralized processing and storage.	Does not by itself define internal module separation.

Final selection: a layered three-tier architecture with a modular backend and optional asynchronous alert processing. It meets current requirements without overengineering the application.

4. Architecture Diagram and Component Design

4.1 Detailed Component Architecture

USER

Enters income, expenses, budgets, and report requests

↓

PRESENTATION LAYER

Login • Dashboard • Transaction forms • Budget screens • Reports

↓

APPLICATION LAYER

Authentication & Access Control | Income & Expense Service | Category Service

Budget & Report Service | Budget Monitoring & Alert Service

↓

DATA ACCESS LAYER

Repository interfaces • Validated queries • Database transactions

↓

RELATIONAL DATABASE

Users • Transactions • Categories • Budgets • Alert history

↓

EXTERNAL NOTIFICATION PROVIDER

Email • Push notifications • In-app alerts (triggered by the Alert Service)

USER

Enters income, expenses, budgets, and report requests

↓

PRESENTATION LAYER

Login | Dashboard | Transaction Forms | Budget Screens | Reports

↓

APPLICATION LAYER

Authentication and Access Control

Income and Expense Service | Category Service

Budget and Report Service | Budget Monitoring and Alert Service

↓

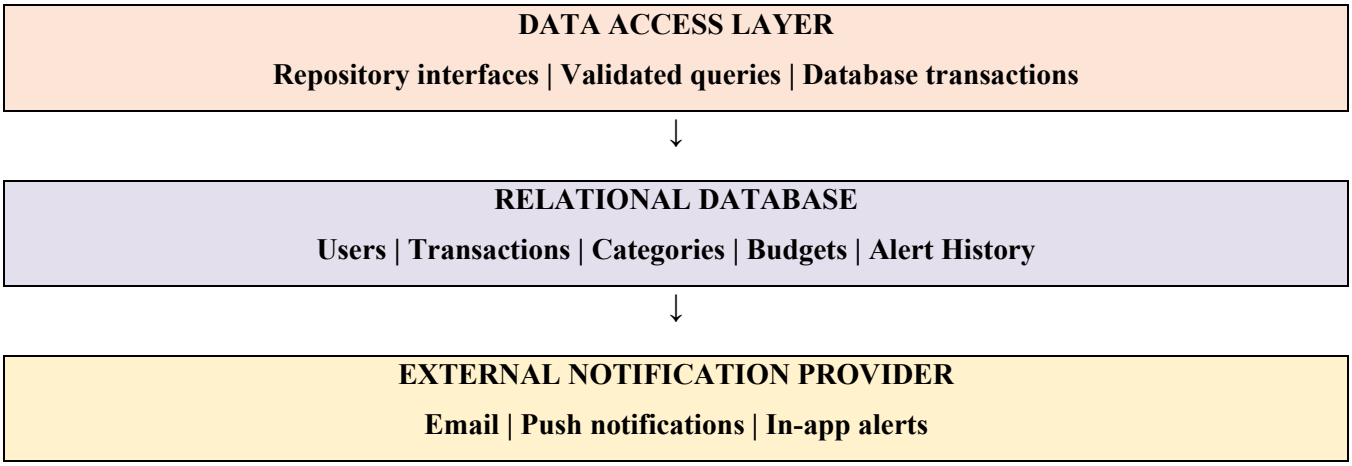


Figure 2. Component-level architecture. The main processing path runs from the user interface through the application and data access layers to the database. The alert service communicates with an external notification provider. In a production implementation, alert delivery may be asynchronous.

4.2 Component Responsibilities

Component	Responsibility	Input and output
User Interface	Collects input and displays dashboards, forms, alerts, and reports.	User actions → API requests and displayed results.
Authentication Service	Verifies credentials and manages authenticated sessions or tokens.	Credentials → authenticated identity or access denial.
Income and Expense Service	Validates and manages financial transactions.	Transaction details → saved or updated record.
Category Service	Creates and maintains income and expense categories.	Category details → category records.
Budget Service	Creates budgets and calculates spending against limits.	Budget period, limits, transactions → budget status.
Alert Service	Evaluates thresholds and coordinates notifications.	Budget status → alert records and notification requests.
Report Service	Aggregates financial data into summaries, tables, and charts.	Date range and filters → reports.
Data Access Layer	Executes database queries and manages persistence.	Repository operations → stored or retrieved records.
Database	Persists users, transactions, categories, budgets, and alerts.	Database operations → durable records and query results.
Notification Provider	Delivers email or push notifications.	Notification request → delivery status where supported.

4.3 Suggested Technology Stack

Layer or component	Suggested technology	Reason
Frontend	HTML, CSS, JavaScript, or React	Builds interactive forms and dashboards.
Backend	Python with FastAPI or Django	Implements APIs, validation, and business logic.
Database	PostgreSQL or MySQL	Supports structured financial records and relationships.
Charts and reports	Chart.js or similar library	Visualizes spending trends and budget usage.
Notifications	Email service or web-push provider	Delivers budget alerts.
Security	HTTPS, secure password hashing, access checks	Protects accounts and financial data.
Testing	Pytest or equivalent framework	Tests business rules, APIs, and integration.
Deployment	Docker and a suitable cloud or server environment	Supports repeatable deployment and scaling.

These are suggested technologies, not mandatory requirements. The final selection should consider the development team's skills, budget, hosting environment, and maintenance needs.

5. Component Interaction and System Workflow

The components work together to ensure that transactions are stored correctly, budgets are updated, alerts are generated when necessary, and reports reflect the latest available data.

5.1 Workflow for Recording an Expense

1. User enters an expense: The user enters ₹800, selects Food, and chooses the transaction date.
2. Authentication and validation: The backend verifies the user's identity, validates the amount, and checks category ownership.
3. Save transaction: The expense service sends the validated record to the data access layer, which stores it in the database.
4. Recalculate budget usage: The budget service totals relevant expenses for the current budget period.
5. Evaluate alert threshold: The alert service checks whether updated spending crosses a configured threshold.
6. Update dashboard: The interface displays the saved expense, remaining budget, and any relevant alert.

5.2 Budget Calculation Example

Assume the user's food budget is ₹5,000 per month and food expenses before the new transaction total ₹3,600.

Calculation	Amount
Monthly food budget	₹5,000
Previous food expenses	₹3,600
New expense	₹800
Total expenses after recording	₹4,400
Remaining budget	₹600
Budget utilization	88%

Budget utilization = (Total Expenses ÷ Budget Limit) × 100

Budget utilization = (₹4,400 ÷ ₹5,000) × 100 = 88%

If the user configures an alert at 80%, the application identifies that the threshold has been crossed and generates a notification. The system should avoid repeatedly sending the same threshold alert for every later transaction unless that behavior is configured.

5.3 Interaction Between Components

From component	To component	Interaction
User Interface	Authentication Service	Sends login or session-verification requests.
User Interface	Transaction Service	Submits expense or income details.
Transaction Service	Category Service	Validates the selected category.
Transaction Service	Data Access Layer	Stores or updates a transaction.
Budget Service	Data Access Layer	Retrieves budgets and relevant transactions.
Budget Service	Alert Service	Supplies updated budget status and threshold information.
Report Service	Data Access Layer	Requests aggregated transaction data.
Alert Service	Notification Provider	Requests delivery of an alert.
Backend	User Interface	Returns transaction status, budget values, and report data.

5.4 Database Design

A relational database is suitable because the application contains related records that must be queried and updated consistently. The following tables represent a proposed logical database schema.

Table	Important fields	Purpose
Users	user_id (PK), name, email, password_hash, created_at	Stores account information and protected credentials.
Categories	category_id (PK), user_id (FK), category_name, category_type	Stores user-defined or system-provided categories.
Transactions	transaction_id (PK), user_id (FK), category_id (FK), amount, transaction_type, transaction_date, description	Stores income and expense records.
Budgets	budget_id (PK), user_id (FK), category_id (FK), budget_amount, start_date, end_date, alert_threshold	Stores budget limits and monitoring settings.
Alerts	alert_id (PK), user_id (FK), budget_id (FK), alert_type, created_at, delivery_status	Records generated alerts and delivery status.

PK means Primary Key; FK means Foreign Key. The Users table links to transactions and budgets through user_id. Transactions reference categories, and budgets can reference the categories they monitor.

Important database rules:

- Amounts must be positive, with income or expense represented separately by transaction_type.
- A transaction's user must own the referenced category.
- Budget periods must have valid start and end dates.
- Database constraints should protect referential integrity.
- Monetary values should use an exact decimal type rather than floating-point arithmetic.
- Queries should be restricted by the authenticated user's identity.
- Indexes should be considered for user IDs, transaction dates, category IDs, and budget periods.

Related changes should use database transactions where appropriate. For example, saving a transaction and recording its processing state should not leave partially completed data if an operation fails.

6. Quality Attribute Analysis

Quality attribute	Architectural approach	Expected result
Security	Secure authentication, authorization, input validation, HTTPS, password hashing, and user-level access restrictions.	Reduces data exposure and unauthorized transactions.
Performance	Database indexing, efficient queries, pagination, and suitable caching.	Improves dashboard and search response times.
Scalability	Stateless API instances, connection pooling, optimized queries, and optional background workers.	Supports increasing users and transaction volumes.
Reliability	Database transactions, validation, backups, retry policies, and error handling.	Reduces corruption and improves recovery.
Availability	Health monitoring, restart policies, redundant hosting where needed, and tested recovery procedures.	Reduces service interruptions.
Maintainability	Clear layer boundaries, modular services, documented interfaces, and automated tests.	Makes updates and debugging easier.
Usability	Responsive design, clear navigation, readable charts, and helpful error messages.	Makes the system easier to use.
Data integrity	Referential constraints, decimal monetary fields, and consistent budget calculations.	Keeps financial reports accurate.
Testability	Unit, API, integration, and security tests.	Helps detect defects before deployment.

6.1 Security Considerations

Financial records are sensitive personal information. The architecture should apply security controls at several levels rather than depending only on the login screen.

- Store passwords using an appropriate password-hashing algorithm; never store plaintext passwords.
- Use HTTPS to protect data transmitted between the client and server.
- Enforce authorization for every operation that accesses a user's transactions or budgets.
- Validate and normalize all input received through APIs.

- Use parameterized database queries or a secure ORM to reduce SQL injection risks.
- Keep credentials and secret keys out of source code.
- Maintain appropriate logs while avoiding unnecessary recording of sensitive financial details.
- Back up the database securely and test recovery procedures.

These practices align with the NIST Secure Software Development Framework (SSDF), which recommends incorporating security requirements and risk mitigation into the software development lifecycle.

6.2 Performance and Scalability

As transaction volume increases, generating financial reports can become expensive. The system should filter transactions by date and category, use database indexes, and perform aggregation in the database when appropriate. Frequently requested summaries may be cached, provided that updates invalidate or refresh stale values.

The first implementation can run as a single deployable backend application. If usage grows, the application can add backend instances behind a load balancer and move notification processing to background workers. This provides a gradual scaling path without introducing microservices too early.

7.2 Advantages and Limitations

Advantages	Limitations and mitigation
Easy to understand and implement.	Poorly defined layers can become tightly coupled; enforce clear interfaces.
Suitable for small and medium-sized applications.	A single backend may become a bottleneck; optimize queries and scale instances.
Business rules can be tested separately.	Integration bugs can still occur; use automated integration testing.
Supports consistent database transactions.	Long-running reports may affect performance; optimize or process them asynchronously.
Lower initial deployment complexity than microservices.	Modules cannot be deployed independently by default; extract services only when justified.
Supports future web and mobile clients.	API compatibility must be maintained as versions change.

7.3 Future Enhancements

7. Automated transaction categorization using machine learning to suggest categories based on descriptions.
8. Bank statement import with duplicate detection and user confirmation.
9. Recurring transactions and reminders for rent, subscriptions, and regular bills.
10. Advanced analytics for spending trends, monthly comparisons, savings ratios, and forecasts.
11. Multi-device synchronization across web and mobile applications.
12. Multi-currency support with clear conversion records.
13. Configurable email, push, and in-app notifications with delivery history.
14. Cloud deployment with managed databases, monitoring, automated backups, and additional application instances when required.

Enhancements should be introduced according to user needs, security requirements, and available resources.

8. Testing and Validation Strategy

Testing verifies that the architecture and its components satisfy functional and non-functional requirements.

Testing type	Example test	Expected outcome
Unit testing	Calculate remaining budget after a new expense.	Correct remaining amount is returned.
Integration testing	Save a transaction and retrieve the updated budget.	Database and application components work together.
API testing	Submit an expense with a negative amount.	Request is rejected with a suitable validation error.
Security testing	Attempt to access another user's transaction.	Access is denied.
Performance testing	Request reports with a large transaction history.	Response time meets the defined target under test conditions.
Reliability testing	Simulate notification-provider failure.	Expense remains stored and retry is handled correctly.
Usability testing	Ask users to record an expense and create a budget.	Users can complete common operations clearly.
Recovery testing	Restore the database from a backup.	Records are recovered within defined recovery objectives.

9. Conclusion

The Personal Expense and Budget Management System demonstrates how architectural design principles can be applied to a practical software engineering problem. The proposed layered three-tier architecture separates user interaction, business processing, and data persistence, making the application easier to develop, maintain, test, and extend.

The system supports income and expense recording, category management, budget creation, threshold alerts, and financial reporting. A relational database maintains structured financial records, while security controls, transaction management, monitoring, and backup procedures support data protection and reliability.